Retrieving Data

Agenda

Select statement
Grouping and Aggregation Functions
Join (**Inner - Left - Right - Full - Cross JOINS)**

Aliases

- SQL aliases are used to give a table, or a column in a table, a temporary name.
- Aliases are often used to make column names more readable.
- An alias only exists for the duration of that query.
- An alias is created with the AS keyword.

- Aliases can be useful when:
  - There are more than one table involved in a query
  - Functions are used in the query
  - Column names are big or not very readable
  - Two or more columns are combined together

Aliases

- Syntax
  - SELECT column_name AS alias_name FROM table_name;
  - SELECT column_name(s) FROM table_name AS alias_name;

```
SELECT name AS employee_name, salary AS employee_salary
FROM employees;
```

Aliases

Example (Table Alias)

```
SELECT e.name, d.name AS department
FROM employees e
JOIN departments d ON e.dep_id = d.id;
```

2. COALESCE

- The COALESCE function returns the first of its arguments that is not null. Null is returned only if all arguments are null. It is often used to substitute a default value for null values when data is retrieved for display
- Syntax

  COALESCE(field1,field2,…)

```sql
SELECT name, COALESCE(salary, 0) AS salary
FROM employees;
```

If salary is NULL → it returns **0**.

```sql
SELECT customerid,
       COALESCE(shipregion,'N/A') AS region
FROM orders;
```

- If shipregion **has a value** → return it
- If shipregion **is NULL** → return 'N/A'

3. ANY

- The ANY operator:
  - returns a boolean value as a result
- returns TRUE if ANY of the subquery values meet the condition
- ANY means that the condition will be true if the operation is true for any of the values in the range.

```sql
SELECT name, salary
FROM employees
WHERE salary > ANY (
    SELECT salary FROM employees WHERE dep_id = 2
);
```

Employees earning more than **any employee in department 2**:

4. ALL

- The ALL operator:
- returns a boolean value as a result
- returns TRUE if ALL of the subquery values meet the condition
- ALL means that the condition will be true only if the operation is true for all values in the range.

```sql
SELECT name, salary
FROM employees
WHERE salary > ALL (
    SELECT salary FROM employees WHERE dep_id = 2
);
```

Employees earning more than **all employees in department 2**:

5. Aggregate Functions

- An aggregate function performs a calculation on a set of values, and returns a single value.
- Most common functions:
  - COUNT()
  - COUNT(DISTINCT)
  - AVG()
  - SUM()
  - MAX()
  - MIN()
  - GROUP_CONCAT()

5. Aggregate Functions

- An aggregate function performs a calculation on a set of values, and returns a single value.
- Most common functions:
  - COUNT()
  - COUNT(DISTINCT)
  - AVG()
  - SUM()
  - MAX()
  - MIN()
  - GROUP_CONCAT()

5. Aggregate Functions

COUNT() Function

- The COUNT() function returns the number of rows that matches a specified criterion.
- Syntax SELECT COUNT(column_name) FROM table_name WHERE condition;

```
SELECT COUNT(*) AS total_employees
FROM employees;
```

5. Aggregate Functions

- The COUNT() function returns the number of different values.
- Syntax

SELECT COUNT(DISTINCT column_name) FROM table_name WHERE condition;

```
SELECT COUNT(DISTINCT Country) FROM Customers;
```

5. Aggregate Functions

The SUM()
function returns the total sum of a numeric column.

- Syntax

SELECT SUM(column_name) FROM table_name WHERE condition;

```
SELECT SUM(salary) AS total_salary
FROM employees;
```

5. Aggregate Functions

The AVG()
function returns the average value of a numeric column.

- Syntax

SELECT AVG(column_name) FROM table_name WHERE condition;

```
SELECT AVG(salary) AS average_salary
FROM employees;
```

5. Aggregate Functions

The MIN()
function returns the smallest value of the selected column.

- Syntax

SELECT MIN (column_name) FROM table_name WHERE condition;

```
SELECT MIN(salary) AS lowest_salary
FROM employees;
```

5. Aggregate Functions

- The MAX()

function returns the largest value of the selected column.

- Syntax

SELECT MAX (column_name) FROM table_name WHERE condition;

```
SELECT MAX(salary) AS highest_salary
FROM employees;
```

6. GROUP BY Statement

- The GROUP BY statement groups rows that have the same values into summary rows.
- The GROUP BY statement is often used with aggregate functions to group the result-set by one or more columns.

Syntax
SELECT column_name(s) FROM table_name WHERE condition GROUP BY column_name(s);

```
SELECT dep_id, AVG(salary)
FROM employees
GROUP BY dep_id;
```

6. GROUP BY Statement

GROUP BY with Multiple Columns in PostgreSQL

```
SELECT dep_id, name, COUNT(*) AS employee_count
FROM employees
GROUP BY dep_id, name;
```

Meaning

- Groups rows by **department and employee name**
- Counts employees in each **dep_id + name combination**

7. Having

- The HAVING clause was added to SQL because the WHERE keyword cannot be used with aggregate functions.
- Syntax

SELECT column_name(s) FROM table_name WHERE condition GROUP BY column_name(s) HAVING condition;

```
SELECT dep_id, AVG(salary)
FROM employees
GROUP BY dep_id
HAVING AVG(salary) > 5000;
```

8. Grouping Sets

More complex grouping operations than those described above are possible using the concept of grouping sets. The data selected by the FROM and WHERE clauses is grouped separately by each specified grouping set

- Syntax

GROUP BY GROUPING SETS ((field1),(field2),(field3,field4));

```
SELECT dep_id, name, SUM(salary) AS total_salary
FROM employees
GROUP BY GROUPING SETS (
    (dep_id),
    (name),
    ()
);
```

8. Grouping Sets

Meaning of Each Group

(dep_id)

Group by department

dep_id	name	total_salary
1	NULL	11000
2	NULL	13500

()

Grand total

dep_id	name	total_salary
NULL	NULL	24500

(name)

Group by employee name

dep_id	name	total_salary
NULL	Ali	11500
NULL	Sara	6000
NULL	Omar	7000

Final Combined Result

dep_id	name	total_salary
1	NULL	11000
2	NULL	13500
NULL	Ali	11500
NULL	Sara	6000
NULL	Omar	7000
NULL	NULL	24500

9. Rollup

Creates hierarchical totals (subtotals + grand total).

```
SELECT dep_id, COUNT(*)
FROM employees
GROUP BY ROLLUP(dep_id);
```

```
ROLLUP ( e1, e2, e3, ... )
```

Same as

```
GROUPING SETS (
    ( e1, e2, e3, ... ),
    ...
    ( e1, e2 ),
    ( e1 ),
    ( )
)
```

Results:
- employees per department
- total employees

10. CUBE

Generates all possible combinations of grouping.

```
SELECT dep_id, COUNT(*)
FROM employees
GROUP BY CUBE(dep_id);
```

```
CUBE ( a, b, c )
```

is equivalent to

```
GROUPING SETS (
    ( a, b, c ),
    ( a, b    ),
    ( a,    c ),
    ( a       ),
    (    b, c ),
    (    b    ),
    (       c ),
    (         )
)
```

Produces:
- per department
- overall total

11. ORDER BY Keyword

- The ORDER BY keyword is used to sort the result-set in ascending or descending order.
- The ORDER BY keyword sorts the records in ascending order by default. To sort the records in descending order, use the DESC keyword.
- Syntax

SELECT column1, column2, ... FROM table_name ORDER BY column1, column2, ... ASC|DESC;

```
SELECT name, salary
FROM employees
ORDER BY salary DESC;
```

11. ORDER BY Keyword

Multi ORDER BY means sorting the result **by more than one column.**

PostgreSQL sorts:
1. First by the **first column**
2. If there are ties, it sorts by the **second column**
3. Then the **third column**, etc.

```
SELECT id, name, dep_id, salary
FROM employees
ORDER BY dep_id ASC, salary DESC;
```

id	name	dep_id	salary
1	Ali	1	5000
2	Sara	1	6000
3	Omar	2	6000
4	Ahmed	2	5000

Meaning
1. Sort employees by **department (ascending)**
2. Inside each department → sort by **salary (highest first)**

11. ORDER BY Keyword

NULLS FIRST and NULLS LAST control **where NULL values appear when sorting results with ORDER BY.**

Normally:
- In **ASC (ascending)** order → `NULL` values appear **last**
- In **DESC (descending)** order → `NULL` values appear **first**

```sql
SELECT name, salary
FROM employees
ORDER BY salary ASC NULLS FIRST;
```

```sql
SELECT name, salary
FROM employees
ORDER BY salary ASC NULLS LAST;
```

12. LIMIT

The LIMIT clause is used to specify the number of records to return.

- The LIMIT clause is useful on large tables with thousands of records. Returning a large number of records can impact performance.

Syntax

SELECT column_name(s) FROM table_name WHERE condition LIMIT number;

```
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 5;
```

13. OFFSET

OFFSET **skips a number of rows before returning results.**

```
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3 OFFSET 2;
```

Meaning

1. Sort by salary descending
2. **Skip the first 2 rows**
3. Return the **next 3 rows**

14. FILTER

FILTER is a PostgreSQL feature used with **aggregate functions** to apply a **condition to the aggregation.** It lets you calculate **different aggregates in one query without using CASE statements.**

aggregate_function(column) FILTER (WHERE condition)

```
SELECT
COUNT(*) FILTER (WHERE salary > 5000) AS high_salary_employees
FROM employees;
```

```
SELECT
COUNT(*) FILTER (WHERE salary >= 6000) AS high_salary,
COUNT(*) FILTER (WHERE salary < 6000) AS low_salary
FROM employees;
```

15. CONCAT_WS() Function

The CONCAT_WS() function adds two or more expressions together with a separator.

- Syntax CONCAT_WS(separator, expression1, expression2, expression3,...)

WS = *With Separator*

```
SELECT CONCAT_WS(' - ', e.name, d.name) AS employee_department
FROM employees e
JOIN departments d ON e.dep_id = d.id;
```

16. ROUND() Function

The ROUND() function rounds a number to a specified number of decimal places.

- Syntax

ROUND(number, decimals)

```
SELECT ROUND(AVG(salary), 2) AS avg_salary
FROM employees;
```

JOINS

Inner - Left - Right - Full - Cross JOINS

SQL JOIN

Before learning **JOIN operations**, we will create a small database that represents a **Student Course System**.
This system will help us understand how tables relate to each other and how JOIN works.

System Scenario
We have a small training center that needs to store:

- Students
- Courses
- Student Enrollments in courses

Each student can enroll in multiple courses.

SQL JOIN

Creating tables first:

```
CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    student_name VARCHAR(50)
);
CREATE TABLE courses (
    course_id SERIAL PRIMARY KEY,
    course_name VARCHAR(50)
);
CREATE TABLE enrollments (
    enrollment_id SERIAL PRIMARY KEY,
    student_id INT,
    course_id INT,
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

SQL JOIN

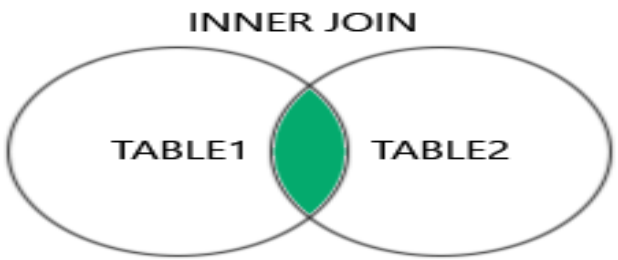
Then Add data to tables:

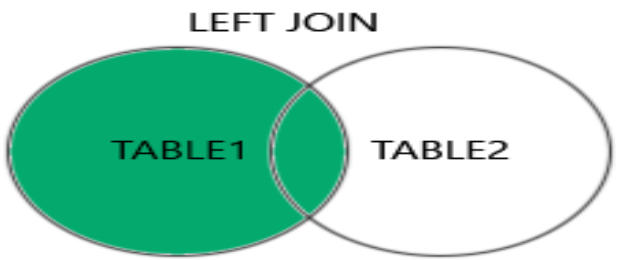
```
INSERT INTO students (student_name) VALUES
('Ali'),('Sara'),('Omar'),('Ahmed');

INSERT INTO courses (course_name) VALUES
('Database'),('JS'),('HTML');

INSERT INTO enrollments (student_id, course_id) VALUES
(1,1),(1,2),(2,1),(3,3);
```

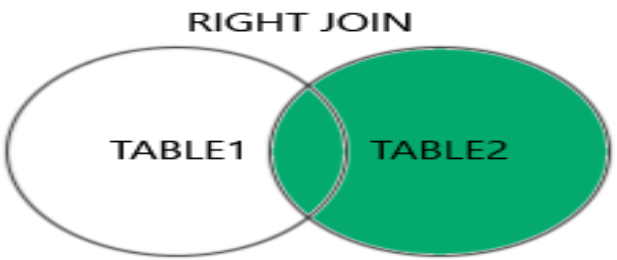
SQL JOIN

A JOIN clause is used to combine rows from two or more tables, based on a related column between them.
Here are the different types of the JOINs in SQL(Most popular used):

- INNER JOIN: Returns records that have matching values in both tables
- LEFT JOIN: Returns all records from the left table, and the matched records from the right table
- RIGHT JOIN: Returns all records from the right table, and the matched records from the left table







SQL JOIN

INNER JOIN - Show all enrolled students and their courses
Shows only students who are enrolled in courses.

Result : only matching rows

```
SELECT s.student_name, c.course_name
FROM enrollments e
INNER JOIN students s
ON e.student_id = s.student_id
INNER JOIN courses c
ON e.course_id = c.course_id;
```

Output:

	student_name character varying (50)	course_name character varying (50)
1	Ali	Database
2	Ali	JS
3	Sara	Database
4	Omar	HTML

SQL JOIN

LEFT JOIN - Show all students, and courses if enrolled Ahmed is shown even though he didn't enroll in any course

Result : Shows all students even if no course.

```
SELECT s.student_name, c.course_name
FROM students s
LEFT JOIN enrollments e
ON s.student_id = e.student_id
LEFT JOIN courses c
ON e.course_id = c.course_id;
```

Output:

	student_name character varying (50)	course_name character varying (50)
1	Ali	Database
2	Ali	JS
3	Sara	Database
4	Omar	HTML
5	Ahmed	[null]

SQL JOIN

RIGHT JOIN – Show all courses, and the students enrolled, even though no student is enrolled in it.

Result : Returns all courses even if no students.

```
SELECT s.student_name, c.course_name
FROM students s
RIGHT JOIN enrollments e
ON s.student_id = e.student_id
RIGHT JOIN courses c
ON e.course_id = c.course_id;
```

Output:

	student_name character varying (50)	course_name character varying (50)
1	Ali	Database
2	Ali	JS
3	Sara	Database
4	Omar	HTML

Thank you